

Operating Systems

An **operating system** is a program which acts as an intermediary between a user and the hardware.

They:

- provide high level abstractions
- hide details of hardware to make application code more portable
- are responsible for resource allocation and management
- control the execution of programs and operation of I/O devices

Dual-mode Operation

- **User mode** - provides a controlled environment for user applications to run in where they have limited access to hardware and resources. Cannot directly run kernel instructions
- **kernel mode** - used when executing privileged instructions, e.g. I/O operations, managing interrupts, halting processes.
- **System calls** - if a process running in user mode needs to execute a privileged instruction, it sends a system call to the OS to do so. These cause a switch to kernel mode so the OS can carry out the instruction before switching back to user mode.
- **mode bit** - 0 for kernel, 1 for user. Provides ability to distinguish between modes.

Multi-programming: keeping multiple jobs in memory. If the one currently executing becomes idle, the CPU is assigned to another.

Multitasking (time sharing): CPU switches between jobs so frequently it appears to be executing them simultaneously - using CPU Scheduling

Operating System Services:

- User interface - GUI or CLI
- Program loading and execution
- I/O operations
- File system manipulation
- Communications - between processes or over networks.
- Error detection
- Resource allocation
- Security.

Processes

A program currently being executed is known as a **process**.

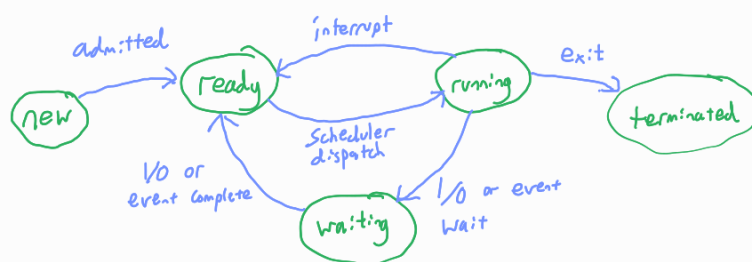
Processes require resources, such as CPU time, memory, I/O and executes its instructions using the resources the OS has allocated to it.

The OS is responsible for process creation, deletion and scheduling

Information about each process is stored in its **process control block (PCB)**, including:

- Process ID
- Process State
- Current values of CPU registers and program counter
- Memory address space allocated to the process
- Priority
- ...

Process State:



Process Scheduling aims to maximise overall CPU use by quickly switching processes onto CPU for multitasking.

The process scheduler selects from 'ready' processes for next execution and maintains:

- Job queue - Set of all processes
- Ready queue - Set of ready processes
- Device queues for processes awaiting I/O

Processes move between the queues as shit happens.

Threads

A thread is a single sequence of instructions that execute within a process. Threads can either be user threads or kernel threads.

Multi-threading is where the OS supports multiple threads within a process. Global variables are allocated to processes, while local variables are per thread. Memory can be either:

Benefits:

- Timing - less time overhead in switching between threads than processes
- Responsiveness - if one thread is blocked, other threads in the same process can continue.
- Resource sharing between threads

Different multi-threading models map user threads to kernel threads in different ways - either 1-1, 1-many, many-many.

Amdahl's Law

$$\text{Speedup} \leq \frac{1}{S + \frac{1-S}{N}}$$

N = number of cores

S = proportion of program that is serial

CPU Scheduling

We want

- Max CPU utilisation
- Max throughput - no. of processes finishing per unit time
- Min turnaround time - time for a process to execute from start to finish, incl. waiting
- Min waiting time
- Min response time - time between request for CPU time and first access to CPU

FCFS

The first process to request access to the CPU is allocated until it finishes.

- Simple to implement
- Waiting time variable
- Short processes have to wait for long boring processes to get out of the way

SJF

Schedule the process with the shortest burst time

- non-preemptive
- Starvation
- FCFS if same length.

SRTF

Preemptive version of SJF - if a new process has less time than the current one has left, swap the new one in.

Round Robin

Give each process a time quantum of CPU time and switch to the next one once the time has elapsed.

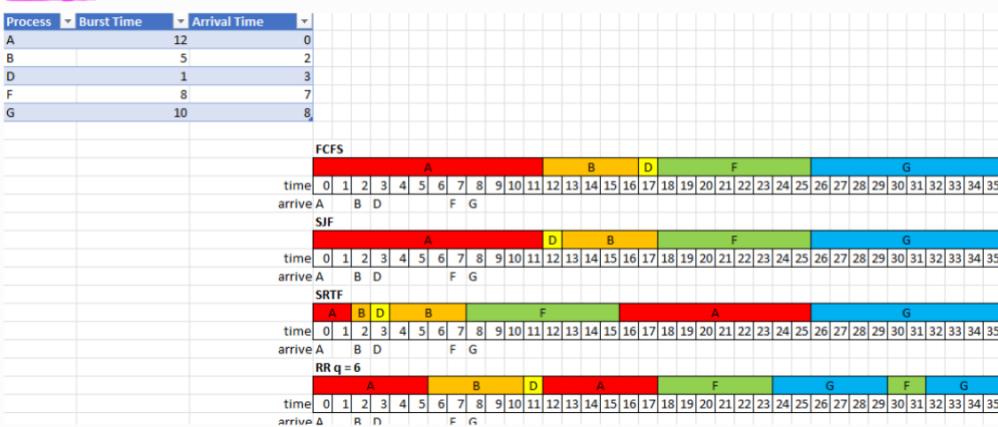
→ Performance depends on the time quantum used - low q means more context switches, high q means equivalent to FCFS

Priority Scheduling

Highest priority first, can be either preemptive or not (as with SJF vs SRTF).

- Smallest number priority = highest priority
- Starvation - low priority processes may never execute
- Solution - ageing - increase priority over time

Example



Memory Management

Each process has a **base register** and **limit register** to define the logical address space for that process.

The memory management unit has to map these virtual addresses to physical addresses in memory.

Memory allocation can be **contiguous**, where processes are allocated memory from a **hole** large enough to accommodate it.

This can be done via

- **first-fit** - allocate from the first hole that's big enough - **fastest**
- **best fit** - allocate smallest hole that's big enough - **best for storage utilisation**
- **worst fit** - allocate biggest hole - **shite**

This contiguous allocation can lead to fragmentation where loads of tiny holes are created that are too small for most processes.

Instead, **paging** is used, where **physical memory** is split into fixed-size blocks called **frames**, and **process' logical memory** is split into blocks of the same size called **pages**.

To run a process, each **page** has to be loaded into a **free frame**. A **page table** manages the mapping between logical and physical addresses.

Virtual memory is where the OS enables programs to address more memory locations than are physically provided in main memory.

Pages are swapped in and out of memory as needed. Each page table entry is given a valid-invalid bit - set to 1 if the page is in physical memory and 0 if not. A **page fault** occurs when a process tries to access a page that isn't actually in memory. If this occurs, the page has to be loaded into a free frame in memory. If all frames are currently occupied, a **page replacement algorithm** must be used to swap out a page already in memory:

FIFO

First in first out

- Suffers from **Belady's anomaly**, where more page faults occur when the number of allocated frames increases for some inputs.

LRU

Least recently used - associates the time of each page's last use and replace the one that was used least recently

- never suffers from Belady's anomaly

LFU

Least frequently used - keep count of how many times each page was used and replace the least used page

MFU

Most frequently used

OPT

Optimal - replace the page that will not be needed for the longest time
- impossible to implement.

Example

Page Reference String:	1	2	3	2	4	1	2	5	1	2	3	4	5
------------------------	---	---	---	---	---	---	---	---	---	---	---	---	---

FIFO (3)	1	1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	2	1	1	1	1	1	3	3	3	
		3	3	3	3	2	2	2	2	2	4	4	

9 page faults

FIFO (4)	1	1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	2	1	1	1	1	5	
		3	3	3	3	3	3	2	2	2	2		
				4	4	4	4	4	4	3	3	3	

10 page faults

LRU (3)	1	1	1	1	4	4	4	5	5	5	3	3	3
	2	2	2	2	2	2	2	2	2	2	2	5	
		3	3	3	1	1	1	1	1	1	4	4	

9 page faults

LFU (3)	1	1	1	1	4	4	4	5	5	5	3	4	5
	2	2	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	1	1	1	1	1	1	1	1	1

9 page faults

MFU (3)	1	1	1	1	1	1	2	5	5	5	5	5	5
	2	2	2	4	4	4	4	4	4	4	4	4	4
		3	3	3	3	3	3	1	2	3	3	3	3

9 page faults

OPT (3)	1	1	1	1	1	1	1	1	1	3	3	3	
	2	2	2	2	2	2	2	2	2	2	4	4	
		3	3	4	4	4	5	5	5	5	5	5	

7 page faults

OPT (4)	1	1	1	1	1	1	1	1	1	1	4	4	
	2	2	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	3	3	3	3	3	3	3	
				4	4	4	5	5	5	5	5	5	

6 page faults

Disk Scheduling

Data stored on hard disks needs to be accessed. To do so, time needs to be spent for the correct sector of the disk - rotational latency - and for the read/write head to move to the correct track - seek time. Disk scheduling algorithms aim to minimise seek time. Requests to pieces of data on disk are maintained by the OS in a queue.

FCFS

First come first serve

- no starvation
- no optimization of seek time

SSTF

Shortest seek time first - selects requests with the minimum seek time from current head position.

- can cause starvation

SCAN

Disk arm moves to one end of the disk servicing requests along the way then bounces back and goes towards the other end

- can cause non-uniform wait times

C-SCAN

As SCAN but instead of rebounding it jumps from one end to the other

LOOK

As SCAN but only goes as far as the first/last request instead of all the way to the end

C-LOOK

Combination of LOOK and C-SCAN.

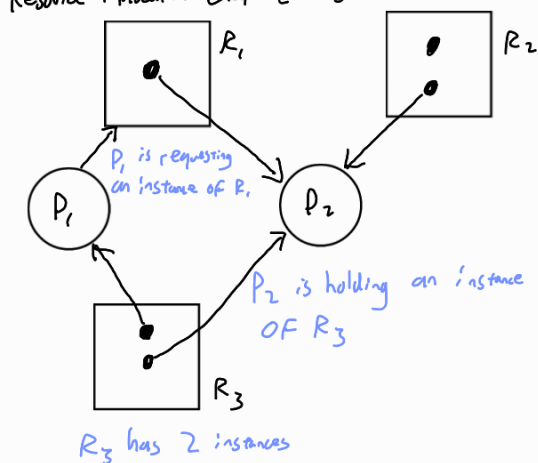
Deadlocks

A set of processes is **deadlocked** if every process in the set is waiting for an event only another process in the set can cause.

A deadlock can occur if all of the following happen simultaneously;

- Mutual exclusion - only one process can use a resource at a time
- Hold and wait - a process is holding a resource and is waiting for resources currently held by another process.
- No preemption - resources cannot be taken from a process unless the process releases it.
- Circular wait - processes depend on each other in a circular fashion

Resource Allocation Graph (RAG)

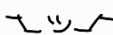


For a deadlock to occur, the corresponding RAG must have a cycle. The presence of a cycle does not necessarily mean deadlock.

A **Safe Sequence** is a sequence that the processes can execute in without creating deadlock. The system is in **Safe State** if such a sequence exists given the resources currently available/being held by processes.

Deadlock avoidance algorithms:

Ostrich

ignore it  - actually best performing

RAG Algorithm

Only grant requests if they don't create a cycle in the RAG.

Banker's Algorithm

Each process must claim maximum use of each resource. When a process requests a resource, it may need to wait. When a process gets all needed resources, they must be returned in finite time.

Let n be the number of processes

m be the number of resources.

Available be a vector of length m , with each value corresponding to the number of available instances of each resource

Max be a matrix of n rows and m columns, representing the max claim of each resource by each process

Allocation be a matrix of n rows and m columns, representing the current allocation of resources to processes

Need = $\text{Max} - \text{Allocation}$

Let $Request_i$ be the request vector for process P_i . (i.e. if $Request_i[j] = k$, P_i wants k instances of R_j). If $Request_i > Need_i$, this is an error as it means the process is going to exceed its maximum claim. Otherwise, if $Request_i > Available$ there isn't enough of the resource available so the process must wait. Else, modify the state by doing:

$$Available = Available - Request_i$$

$$Allocation = Allocation + Request_i$$

$$Need_i = Need_i - Request_i$$

If these changes create an unsafe state, revert back and the process must wait. Otherwise, allocate the requested resources to P_i .

Deadlock recovery:

- Terminate processes until safe state
- Rollback to previous safe state